Specifying Application Services in Docker Compose

In this lesson, we'll learn how to specify application services in a Docker Compose file, setting the foundation for multi-container applications. We'll delve into defining services and how they interact within the application stack.

Learning objective

> - Learn how to specify application services, setting the foundation for multi-container applications.

Introduction to Docker Compose Services

> Managing multiple containers for a single application can be complex. Docker Compose simplifies this by allowing you to define and run multi-container Docker applications using a single YAML file. This lesson will guide you through the process of specifying application services within a Docker Compose file, enabling efficient orchestration of your application stack.

Setting Up the Project Directory

> The first step in any Docker Compose project is to create a dedicated directory. This directory will house all your configuration files, including the docker-compose.yml file and any related application code.
>
> To create a new project directory and navigate into it, use the following commands in your terminal:
>
> ```
> mkdir my_docker_project && cd my_docker_project
> ```
>
> Once inside the directory, you can open it with your preferred code editor (e.g., VS Code) to begin configuring your Docker Compose file:

```
code .
```

Creating the Docker Compose File

Docker Compose relies on a YAML file (typically named docker-compose.yml) to define the services, networks, and volumes that constitute your application. Let's create a basic docker-compose.yml file to define a web service using Nginx:

```
version: '3.8'
services:
web:
image: nginx:latest
ports:
- "8080:80"
```

In this configuration:

- version: '3.8' specifies the Docker Compose file format version.
- services: is the top-level key under which all your application services are defined.
- web: defines a service named "web".
- image: nginx:latest tells Docker Compose to use the latest Nginx Docker image for this service.
- ports: maps port 8080 on your host machine to port 80 inside the Nginx container, making the web service accessible via http://localhost:8080.

Adding a Database Service

Most applications require a database. You can easily extend your docker-compose.yml file to include a database service, such as PostgreSQL. Add the following service definition to your existing docker-compose.yml file, under the services key:

```
db:
image: postgres:latest
environment:
POSTGRES_USER: user
POSTGRES_PASSWORD: password
POSTGRES_DB: mydatabase
```

Here, we define a new service named "db" using the latest PostgreSQL image. The environment section allows you to set environment variables within the container, which are used by PostgreSQL to configure the database user, password, and initial database name.

Running Docker Compose

Once your services are defined in the docker-compose.yml file, you can start your entire application stack with a single command from your project directory in the terminal:

```
docker-compose up -d
```

The -d flag runs the containers in "detached" mode, meaning they will run in the background without occupying your terminal. Docker Compose will pull the necessary images (if not already present), create networks, and start the defined services.

To verify that your services are running, you can use the docker ps command:

```
docker ps
```

This command lists all currently running containers, allowing you to confirm that both your Nginx web service and PostgreSQL database are active.

Testing the Application

After starting your services, it's good practice to test if your web service is accessible. Since we mapped port 8080 on the host to port 80 in the Nginx container, you can access the Nginx welcome page through your web browser.

Open your web browser and navigate to http://localhost:8080/. If your Docker Compose setup is correct, you should see the default Nginx welcome page, confirming that your web service is operational.

Stopping and Removing Containers

When you are finished working with your application, you can stop and remove all the services defined in your docker-compose.yml file using a single command:

```
docker-compose down
```

> This command gracefully stops and removes all containers, networks, and volumes that were created by Docker Compose for your project, helping to clean up your environment.

Conclusion

You have successfully learned how to define and manage multiple application services using Docker Compose. This powerful tool simplifies the orchestration of multi-container applications, allowing you to spin up an entire application stack with just one command. By mastering these foundational concepts, you are well-equipped to manage more complex containerized deployments efficiently.

Key Takeaways ✖

- **Docker Compose Purpose:** Simplifies defining and running multi-container Docker applications using a single YAML file.
- **docker-compose.yml Structure:** Services, networks, and volumes are defined under top-level keys in this YAML configuration file.
- **Service Definition:** Each service specifies an image, ports, environment variables, and other configurations for a container.
- **docker-compose up -d:** Command to build, create, and start all services in detached mode (background).
- **docker-compose down:** Command to stop and remove all containers, networks, and volumes defined in the Compose file.